第5章 第3节 RAG核心原理（二）-检索生成与关键机制

- 来源: https://class.imooc.com/lesson/5271#mid=104915
- 阶段: 阶段二: 【实战入门】大模型开发与知识接入
- 周: 第5周 RAG：知识接入技术
- 模块: RAG核心原理
- 课节: 图文 1-3 RAG核心原理（二）-检索生成与关键机制

1-3 RAG核心原理（二）-检索生成与关键机制

第5章 第3节 RAG核心原理（二）-检索生成与关键机制

3.1 上下文拼接：喂给AI的"参考资料"

RAG系统需要把检索到的文档（比如"年假天数规定"“申请流程”“注意事项”）和用户问题（“我入职2年了，下周想请5天年假，来得及吗？”）整合成一个结构化的Prompt，然后交给AI来回答。

这个Prompt该怎么设计呢?

基本模板

```
你是一个专业的客服助手。请基于以下参考资料回答用户问题。

【参考资料】
文档1：年假天数规定
入职1年内：5天；1-3年：7天；3-5年：10天；5年以上：15天

文档2：年假申请流程
登录OA系统 → 选择"请假申请" → 填写年假类型和时间 → 提交领导审批

文档3：申请注意事项
需要提前3天申请；春节、国庆等重大节假日需提前1个月

【用户问题】
我入职2年了，下周想请5天年假，来得及吗？

【回答要求】
1. 基于参考资料回答，不要编造信息
2. 如果参考资料中没有相关信息，明确告知用户
3. 回答要准确、友好、自然

1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19

预览
```

模板设计直接影响回答质量。对比两种模板：

Bad：

```
资料：[一大堆文档堆砌]
问题：我入职2年了，下周想请5天年假，来得及吗？


1
2
```

AI可能回答：

- 不理解哪些是参考资料，哪些是问题
- 可能忽略关键信息
- 回答格式混乱

Good：

```
你是专业客服。基于【参考资料】回答【用户问题】，遵守【回答要求】。

【参考资料】
...（结构化呈现）

【用户问题】
...

【回答要求】
...（明确约束）
```

AI会：

- 清楚角色定位（专业客服）
- 明确信息边界（只用参考资料）
- 遵守回答规范（准确、友好）

3.2 生成阶段：AI的魔法时刻

当AI收到完整的Prompt后，它会：

理解用户意图

从问题"我入职2年了，下周想请5天年假，来得及吗？"中提取关键信息：

- 入职时长：2年
- 需要天数：5天
- 时间：下周
- 核心问题：来得及吗（时间是否允许）

从参考资料中定位答案

- 从文档1找到：入职2年 → 对应7天年假
- 从文档2了解：申请流程
- 从文档3确认：需提前3天申请

逻辑推理

- 7天年假 vs 需要5天 → 够用 ✓
- 下周出发 vs 提前3天 → 假设今天是周一，距下周一超过3天 → 来得及 ✓

生成回答

综合以上信息，生成自然流畅的回答：

```
可以的，来得及！

根据公司规定：
您入职2年，有7天年假额度，请5天完全够用
普通时段只需提前3天申请，下周去旅游符合要求

建议您现在就登录OA系统提交申请：
1. 选择"请假申请"
2. 填写年假类型和日期（5天）
3. 提交给直属领导审批

祝您旅途愉快！
```

3.3 RAG的关键优势

经过这一节的讲解，我们可以总结出RAG的几个关键优势：

语义理解

不依赖精确的关键词匹配，能理解问题的真实意图。

例如：

- “怎么请假” ≈ “假期申请流程” ≈ “如何提交休假申请”
- RAG能识别这些表述在语义上是相同的

跨文档整合

能从多份文档中提取信息，综合回答。

例如："年假够用吗？“需要结合"天数规定” + “申请流程” + “个人入职时间”

个性化生成

基于用户的具体情况（入职2年、请5天、下周）生成定制化回答，而不是千篇一律的标准答案。

可追溯性

可以标注答案来源于哪些文档，便于验证准确性，增强可信度。这一点很重要，这是RAG可以部分解决AI“幻觉”的关键能力，他可以给出他是基于哪些资料生成的答案，我们可以核查信息真实性。

灵活扩展

添加新文档只需重新向量化并存入数据库，无需修改系统逻辑或重新训练模型。

3.4 技术细节补充

在理解了RAG的整体流程后，这里补充几个实际实现中容易被忽略但很重要的技术细节。

文档切分（Chunking）：检索的其实不是完整文档

你可能会有个疑问：检索到文档后，是直接把整份文档发给AI，还是会提取其中的内容？

*答案是：检索到的根本不是完整文档，而是文档的片段（Chunk）。

为什么要切分

想象一下，公司有一份50页的《员工手册》，里面包含：

- 第1-5页：入职流程
- 第6-10页：薪资福利
- 第11-15页：请假制度
- 第16-20页：报销流程
- ...

如果用户问"怎么申请年假？"，真正相关的只有第11-15页的内容。但如果你把整份50页的手册都塞进Prompt：

问题1：Token超限

- 50页 ≈ 25000字
- 大部分模型上下文窗口有限
- 即使能塞下，成本也非常高

问题2：噪音太多

- AI需要从50页中找到相关的5页
- 大量无关信息会干扰判断
- 回答质量下降

问题3：检索不精准

- 整份文档的向量是所有内容的"平均语义"
- 无法精确匹配到"年假申请"这个具体主题

正确的做法：离线切分

在建立知识库时，就把文档切成小块：

```
原始文档：《员工手册.pdf》（50页，25000字）
          ↓ 切分

Chunk 1: "入职流程：新员工需要..."  (约200字)
Chunk 2: "薪资发放：每月15日..."    (约180字)
Chunk 3: "年假天数：入职1年内..."   (约150字)
Chunk 4: "年假申请流程：登录OA..."  (约220字)
Chunk 5: "年假注意事项：需提前..."  (约160字)
Chunk 6: "报销流程：提交发票..."    (约190字)
...
Chunk 125: "离职手续：需提前..."    (约200字)
```

每个Chunk：

- 转换成向量
- 存入向量数据库
- 保留原始文本

> [!TIP] 第3点"保留原始文本"很重要 我们向AI提交的不能是向量，必须是文本。有时候我们需要追溯原始资料用来判断AI是否出现"幻觉"，这也需要能够看到Chunk的原始文字，人类是看不懂数字向量的。

检索时的实际流程

```
# 用户提问
question = "我怎么申请年假？"

# 检索相似的Chunk（不是完整文档）
top_chunks = vector_db.search(question, top_k=3)
# 返回:
# - Chunk 4: "年假申请流程：登录OA系统 → 选择请假申请..."
# - Chunk 3: "年假天数：入职1年内5天；1-3年7天..."
# - Chunk 5: "年假注意事项：需提前3天申请..."

# 直接把这些Chunk的原始文本拼接到Prompt
prompt = f"""
【参考资料】
{top_chunks[0].text}
{top_chunks[1].text}
{top_chunks[2].text}

【用户问题】
{question}
"""
```

优势

- Token消耗少（3个Chunk约600字 vs 整份文档25000字）
- 检索精准（直接匹配到相关片段）
- 成本降低（Token少了40倍）

常见的切分策略

策略	适用场景	Chunk大小
按字符数切分	通用场景	200-500字
按段落切分	结构化文档	自然段落
按语义切分	技术文档	完整的概念单元
滑动窗口	连续性强的内容	重叠50-100字

所以，流程图中的"步骤3：提取相关片段"并不是从完整文档中提取内容，而是从向量数据库中取出检索到的Chunk的原始文本。文档在存储时就已经切分好了。

Chunk重叠

为了避免重要信息被切断，相邻Chunk通常会有重叠：

```
Chunk 1: "...年假天数规定：入职1年内5天；1-3年7天..."
Chunk 2: "...1-3年7天；3-5年10天。申请流程：登录OA系统..." ← 重叠部分

1
2
```

3.5 元数据

什么是元数据

元数据（Metadata）字面意思是"关于数据的数据"。如果把文档内容比作一本书的正文，那元数据就是这本书的封面信息——书名、作者、出版日期、ISBN号等。

在编程中，元数据无处不在：

- 文件的元数据：文件名、大小、创建时间、修改时间
- 照片的元数据：拍摄时间、地点、相机型号、分辨率
- 数据库记录的元数据：创建人、创建时间、最后修改时间

元数据的作用是帮助我们组织、查找、过滤和管理数据本身。

帮助我们组织、查找、过滤和管理数据本身

在RAG系统中，每个Chunk除了保存文本内容，还会保存元数据：

```
chunk = {
    "text": "年假申请流程：登录OA系统...",
    "metadata": {
        "source": "员工手册.pdf",
        "page": 12,
        "section": "请假制度",
        "last_updated": "2024-01-15"
    }
}
```

3.6 多路召回

什么是召回 "召回"（Recall）是搜索系统中的术语，意思是"从海量数据中把相关的结果找回来"。就像你在图书馆找书，"召回"就是从几万本书里把可能相关的书都拿出来的过程。

"多路"则是指使用多种不同的方法来召回结果，然后把它们合并起来。

多种不同的方法

为什么需要多路召回？因为单一的检索方法都有局限性：

单一的检索方法都有局限性

- 纯向量检索的问题：

```
用户问："公司OA系统的网址是多少？"

向量检索结果：
- "OA系统使用指南..."  （相似度0.82）
- "如何登录OA系统..."  （相似度0.78）
- "OA系统常见问题..."  （相似度0.75）

但都没有直接包含网址！
```

向量检索擅长语义理解，但对精确信息（网址、电话号码、专有名词）可能找不准。

精确信息

- 纯关键词检索的问题：

```
用户问："怎么请假？"

关键词检索：搜索包含"请假"的文档
- "请假制度详细说明..."  ✓
- "员工请假统计表..."    × （不相关）
- "请假审批流程图..."    ✓

无法理解"申请年假""休息几天"等同义表达
```

关键词检索擅长精确匹配，但不理解语义。

精确匹配

语义

- 多路召回的解决方案：

结合两种方法的优势：

```
用户问："公司OA系统的网址是多少？"

# 路径1：向量检索（Top-5）
vector_results = [
    "OA系统使用指南：登录地址为...",
    "如何登录OA系统...",
    "OA系统常见问题..."
]

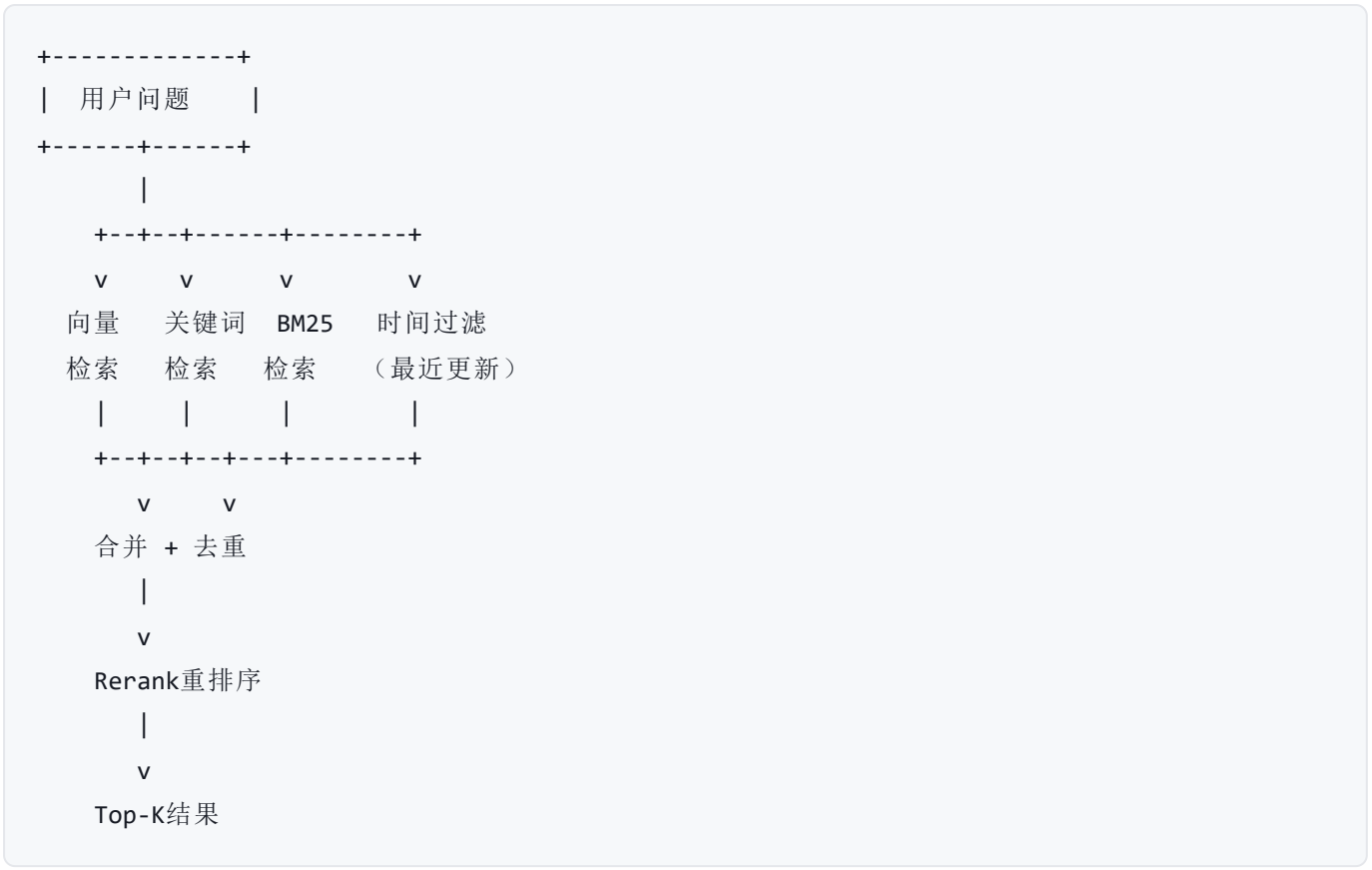
# 路径2：关键词检索（Top-5）
keyword_results = [
    "OA系统网址：https://oa.company.com",  ← 精确命中！
    "内部系统网址汇总：OA系统...",
    "网络资源：OA系统地址为..."
]

# 合并去重
all_results = vector_results + keyword_results
# 结果：共8个候选文档（去重后）

# 重新排序（Rerank）
# 使用更精细的模型对这8个文档重新打分
final_results = rerank(all_results, question)
# 最终Top-3：
# 1. "OA系统网址：https://oa.company.com"  (0.95)
# 2. "OA系统使用指南：登录地址为..."      (0.88)
# 3. "内部系统网址汇总：OA系统..."        (0.82)
```

常见的召回路径组合

> [!TIP] 什么是 BM25？ BM25（Best Match 25）是一种经典的基于关键词的排序算法，由搜索引擎领域发展而来。它和向量检索不同： 向量检索：理解“语义”（例如“请假”和“年假申请”语义相近） BM25：关注“词频”（文档里关键词出现越多、在整个知识库里越稀有，得分越高） 在多路召回中，BM25 常用来作为“精确匹配补充”：当用户问题包含关键专业名词或产品型号时，BM25 能准确抓取到包含这些词的文档。



这就是“多路召回”的核心思想：让不同的检索方法各显神通，最后综合决策。就像组建一个专家团队，每个专家擅长不同领域，共同给出最佳答案。

3.7 下节预告

在这一节中，我们多次提到“Embedding”——把文字转换成向量。但它到底是怎么工作的？

为什么一段文字能变成一组数字？这些数字又是如何表达语义的？不同的 Embedding 模型（768维、1536维、3072维）该如何选择？

下一节《Embedding：把文字变成向量》将带你深入理解这项RAG的关键技术，从原理到实战，掌握如何选择和使用Embedding模型。

3.8 ■ 学点英语

中文	English	音标	说明
上下文拼接	Context Assembly	/ˈkɑːntekst əˈsembli/	将检索到的文档和用户问题整合成结构化Prompt的过程
多路召回	Hybrid Search	/ˈhaɪbrɪd sɜːrtʃ/	同时使用多种检索方法（向量、关键词、BM25）后合并结果
重排序	Rerank	/riːˈræŋk/	对初步检索结果重新打分排序，提升最相关内容的位置
BM25	BM25	/biː em twɛnti faɪv/	基于词频和逆文档频率的经典文本检索算法
引文追溯	Citation Tracking	/saɪˈteɪʃn ˈtrækɪŋ/	要求AI在回答中引用来源，确保答案可验证